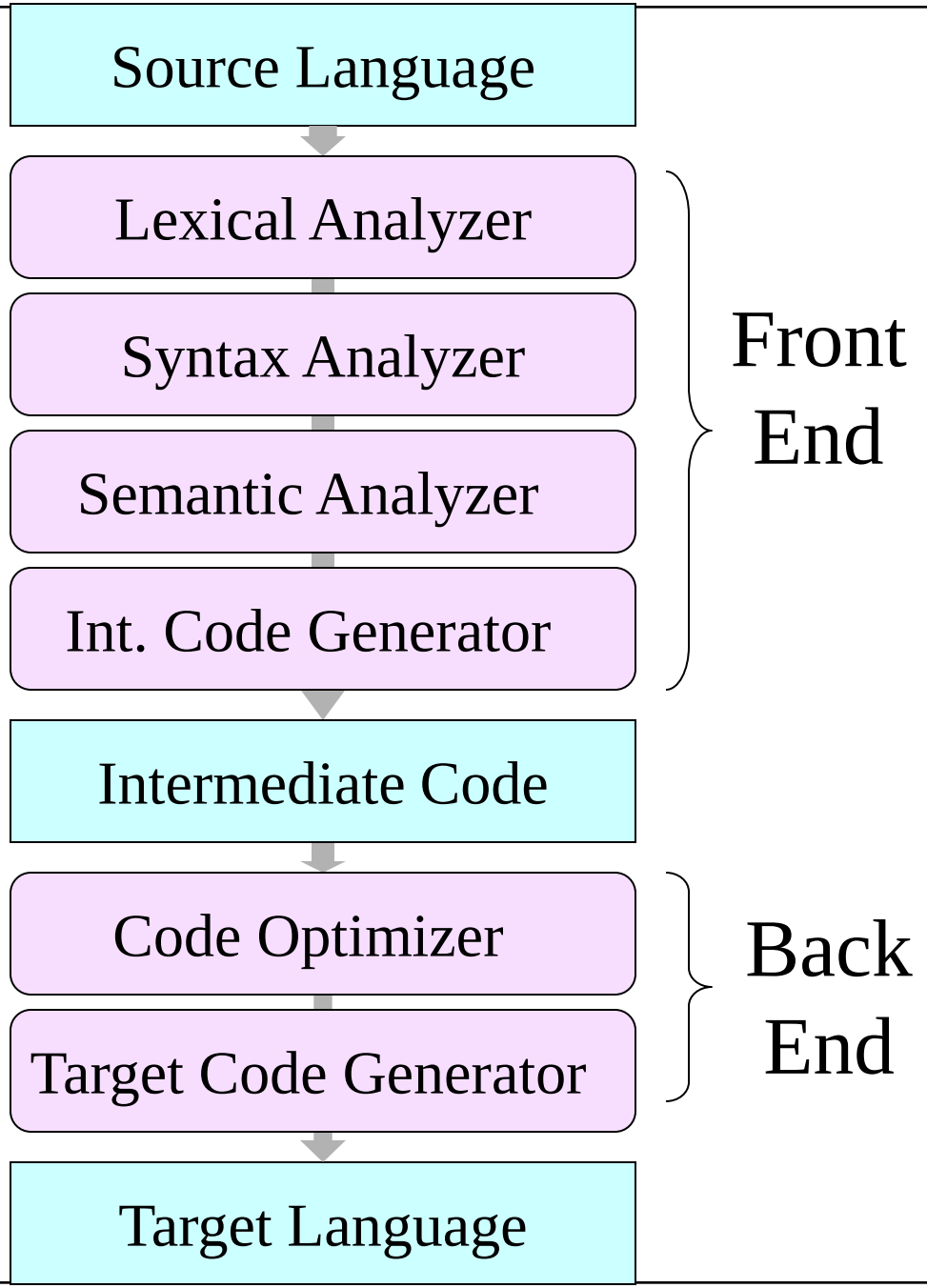
COMPILERS
(LECTURE 2)

By :

Dr. Sherin Omran

Structure of a Compiler

- Front end: Analysis
  - Read source program and understand its structure and meaning
- Back end: Synthesis
  - Generate equivalent target language program



Introduction

- The analysis phase of a compiler breaks up a source program into constituent pieces and produces an internal representation for it, called intermediate code.
- The synthesis phase translates the intermediate code into the target program. Analysis is organized around the \syntax" of the language to be compiled.
- The syntax of a programming language describes the proper form of its programs, while the semantics of the language denes what its programs mean; that is, what each program does when it executes.
- For specifying syntax, we present a widely used notation, called context-free grammars (CFG).

Syntax and Grammar

- Context-Free Grammar (CFG) or "grammar" for short - that is used to specify the syntax of a language.
- Grammars will be used throughout this course to organize compiler front ends.
- A grammar naturally describes the hierarchical structure of most programming language constructs.
- For example, an if-else statement in Java can have the form

  if (expression) statement else statement

Syntax and Grammar

- The rule of statement in the grammer can be described as

  stmt → if (expr) stmt else stmt

- Such a rule is called a production. In a production, lexical elements like the keyword if and the parentheses are called **terminals**.
- Variables like expr and stmt represent sequences of terminals and are called **nonterminals**.

Syntax and Grammar

Context Free Grammar (CFG):

- A set of recursive rewriting rules used to specify the syntax of a language.
- It describes the hierarchical structure of most programming languages.
- It has 4 components G=(N, T, S, P) where, N is the set of non-terminals, T is the set of terminals, S is the start symbol and P is the set of production rules.

Syntax and Grammar

- A grammar *G* is defined as a quadruple *G* =(*V*, *T*, *S*, *P*),

Where:

- *V* is a finite set of objects called **variables**,
- *T* is a finite set of objects called **terminal symbols**,
- $S \in V$ is a special symbol called the **start** variable,
- *P* is a finite set of **productions**.
- A grammar G = (V , T , S , P) is said to be context-free if all productions in P have the form $A \rightarrow x$

  where $A \in V$ and $x \in (V \cup T)^*$.

From CFG to Parse or derivation tree

- Formally, given a context-free grammar, a parse tree according to the grammar
- is a tree with the following properties:

    1. The root is labeled by the start symbol.
    2. Each leaf is labeled by a terminal or by

       empty string (λ or ε)
    3. Each interior node is labeled by a nonterminal.

Syntax and Grammar

- The language defined by the following grammar consists of lists of digits separated by plus and minus signs.

$$
\begin{aligned}
list &\rightarrow list + digit \\
list &\rightarrow list - digit \\
list &\rightarrow digit \\
digit &\rightarrow 0 \mid 1 \mid 2 \mid 3 \mid 4 \mid 5 \mid 6 \mid 7 \mid 8 \mid 9
\end{aligned}
$$

Syntax and Grammar

- The grammar $G = (\{S\}, \{a, b\}, S, P)$, with productions

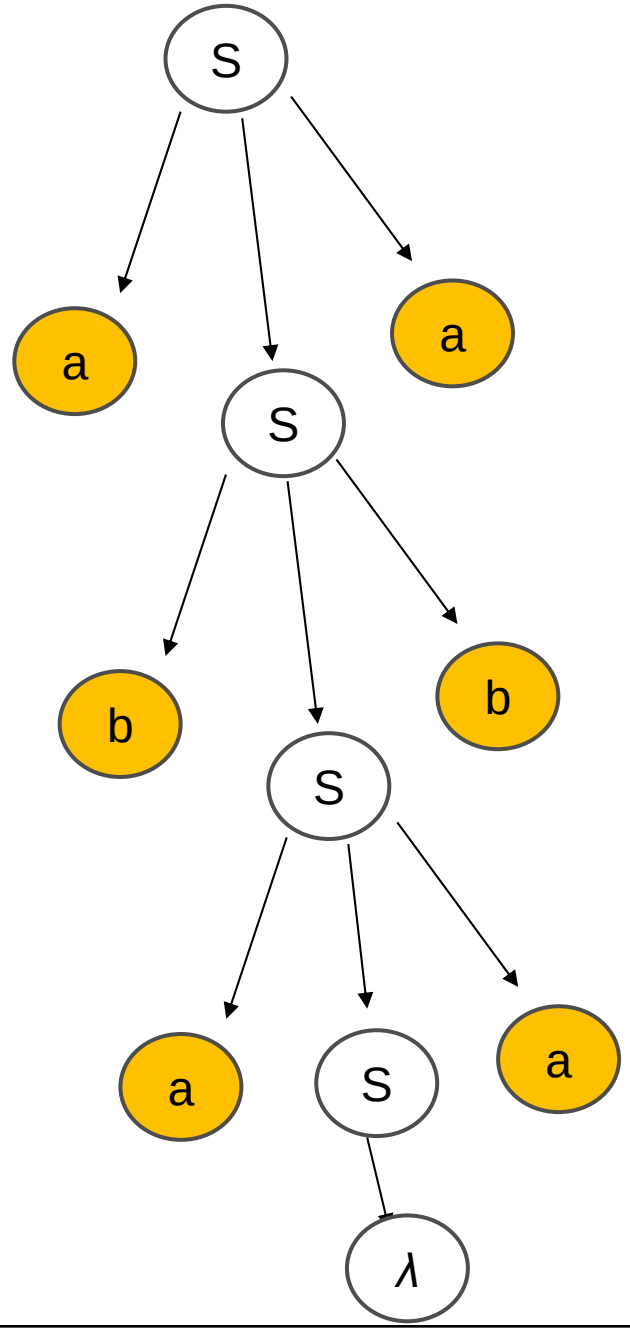
  $S \rightarrow aSa$

  $S \rightarrow bsb$

  $S \rightarrow \lambda$

- Is a context-free grammar
- Show how to derive the expression abaaba using the above grammar:

  S → aSa → abSba → abaSaba →abaaba



Derivation or Parse tree

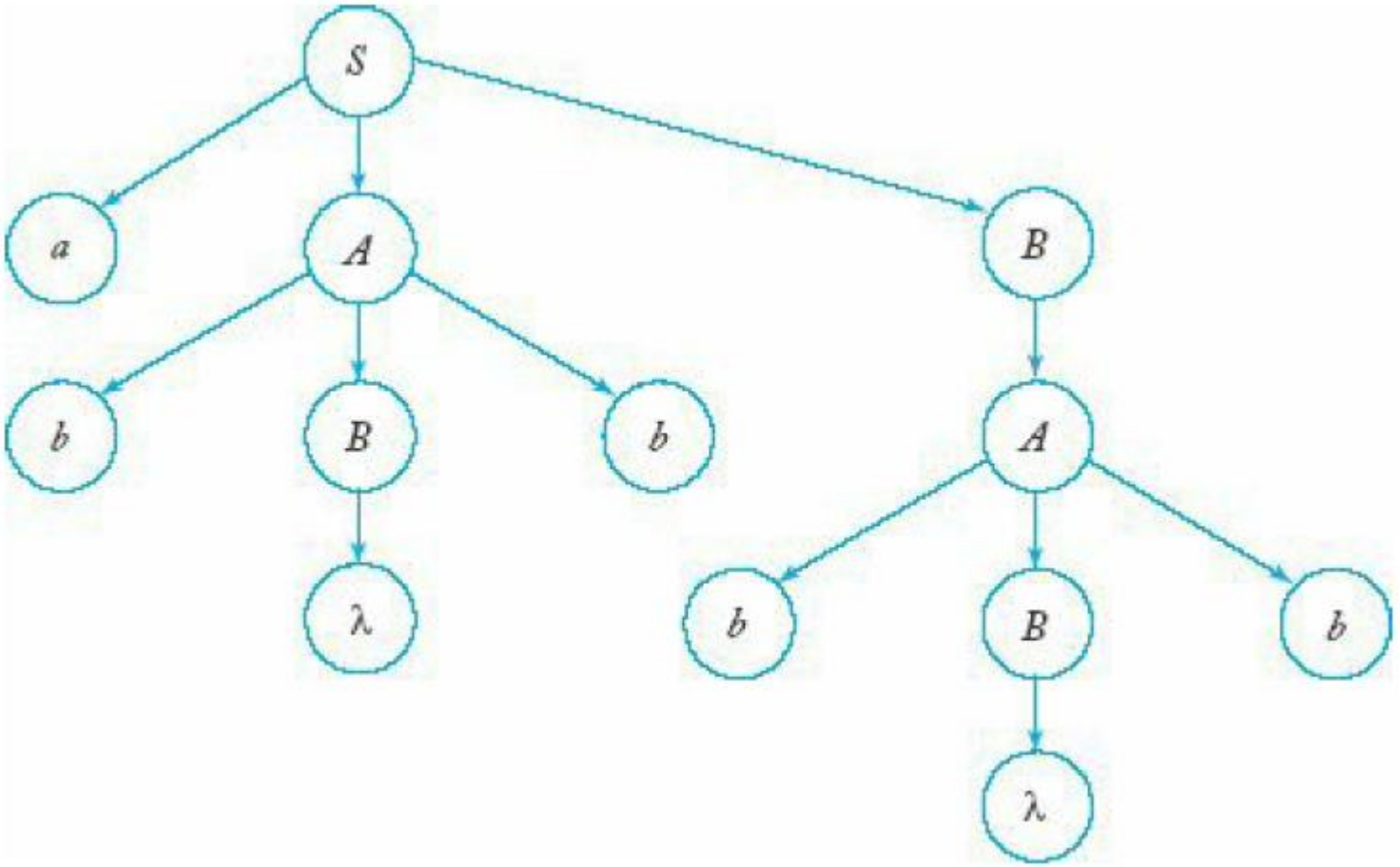
Example

- Given a grammar G with the following productions

$$S \rightarrow aAB,$$
$$A \rightarrow bBb,$$
$$B \rightarrow A|\lambda.$$

- Show the derivation tree for the string *abbbb*
- *S--->a**A**B ---> a**bBb**B--->abb**B**--->abb**A**--->abb**bBb**--->abb**bb**.*

- *S--->a**A**B ---> a**bBb**B--->abb**B**--->abb**A**--->abb**bBb**--->abb**bb***



Another example

- Show a derivation tree for the string aabbbb with the grammar

  S →AB|λ ,

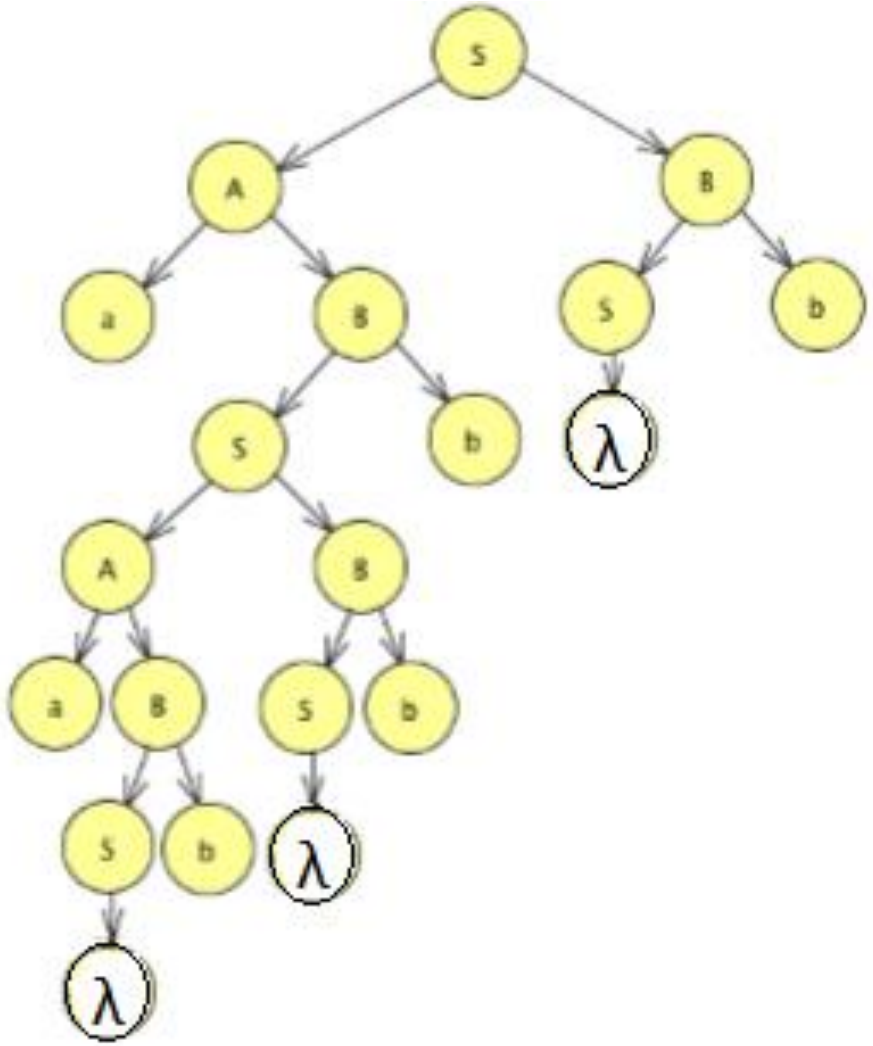
A → aB,

  B → Sb.

Solution :

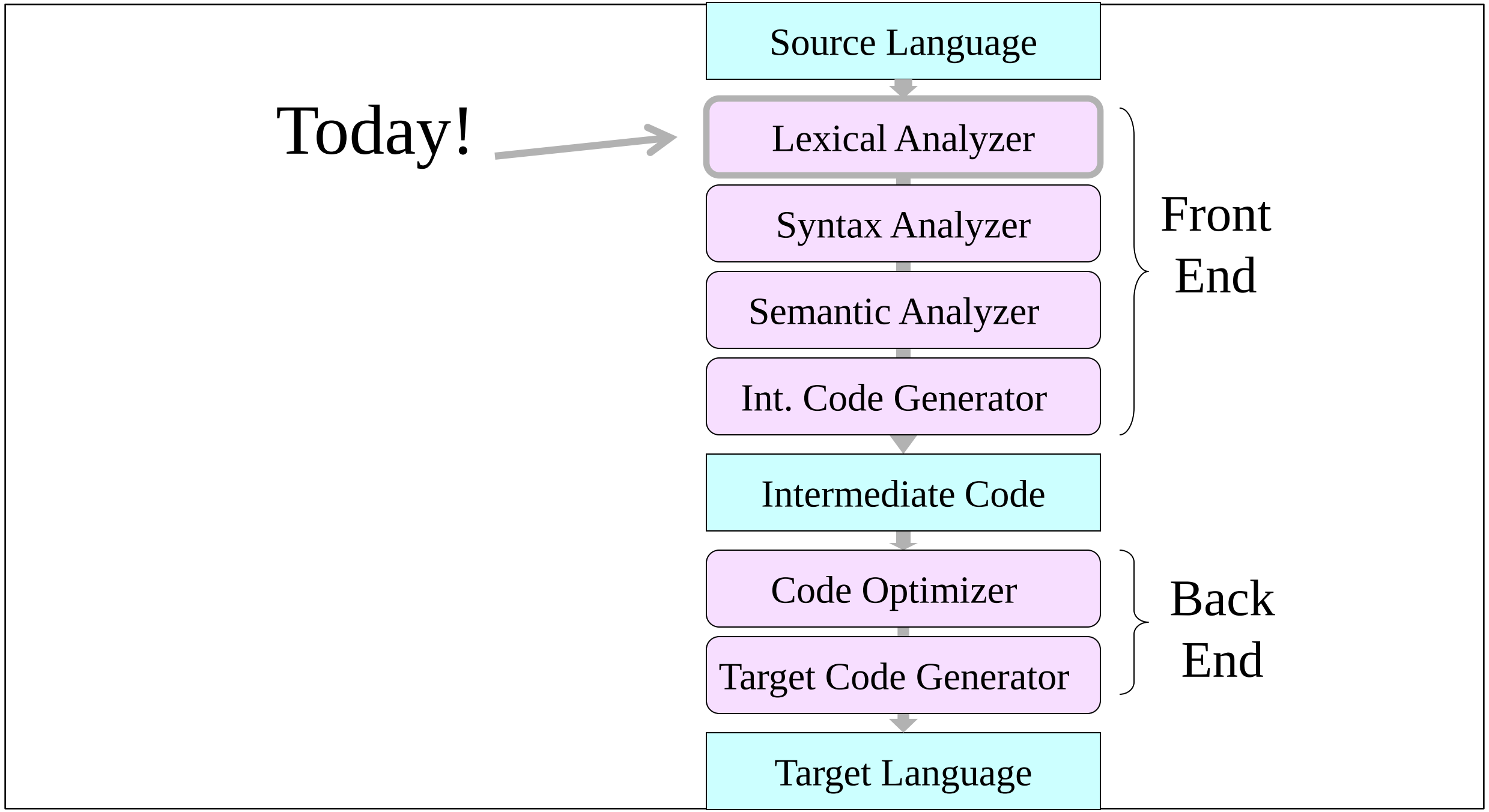
S → AB → aBB → aSbB → aABbB → aaBBbB → aaSbBbB → aabBbB → aabSbbB → aabbbB → aabbbSb → aabbbb

derivation tree for the string aabbbb

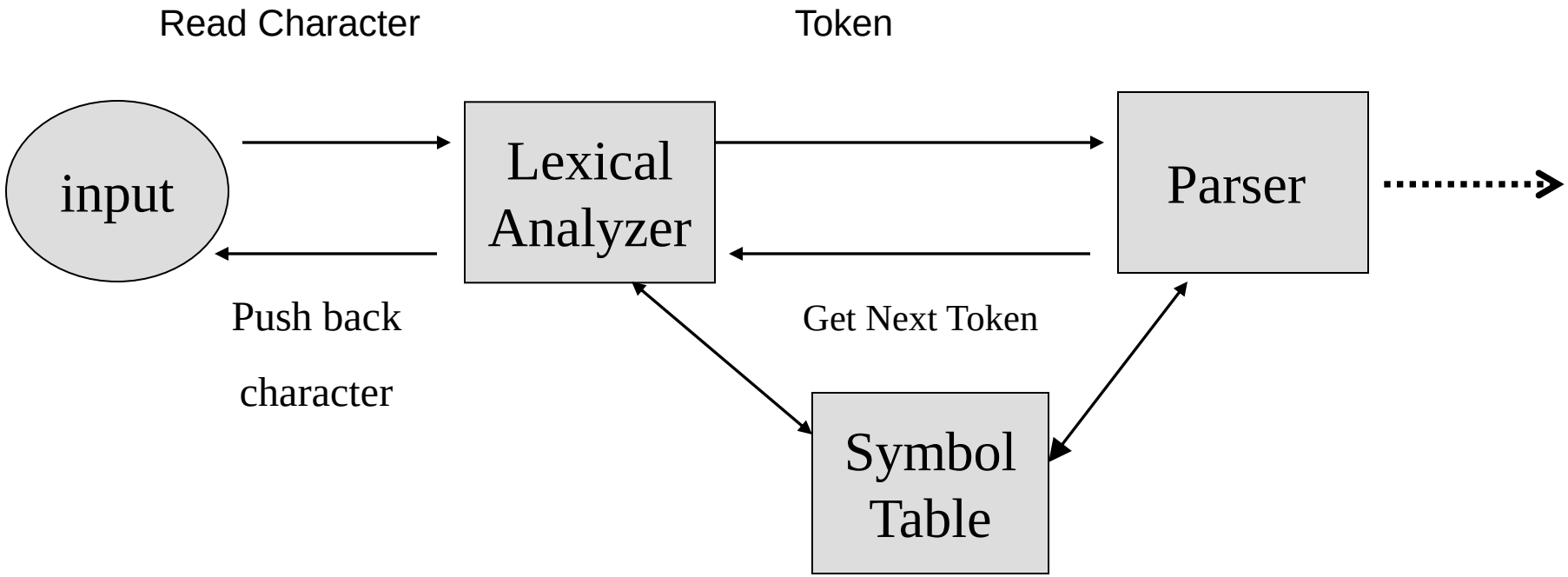


Lexical Analyzer

- The lexical analyzer is the first phase of a compiler.
- The main task of lexical Analyzer (Scanner) is to read a stream of characters as an input and produce a sequence of tokens that the parser (Syntax Analyzer) uses for syntax analysis.


Today!
Source Language
Lexical Analyzer
Syntax Analyzer
Semantic Analyzer
Int. Code Generator
Front End
Intermediate Code
Code Optimizer
Target Code Generator
Back End
Target Language

The Role of Lexical Analyzer (cont'd)



The interaction between the lexical analyzer and the parser

The Role of Lexical Analyzer (cont'd)

- When a lexical analyzer is inserted b/w the parser and input stream ,it interacts with two in the manner shown in the previous fig.
- It reads characters from the input ,groups them into lexeme and passes the tokens formed by the lexemes, together with their attribute value, to the parser.
- In some situations ,the lexical analyzer has to read some characters ahead before it can decide on the token to be returned to the parser.

The Role of Lexical Analyzer (cont'd)

The main task of the lexical analyzer is to:

1- Read the input characters of the source program,

2- Group them into lexemes,

3- Produce as output a sequence of tokens for each lexeme in the source program.

The stream of tokens is sent to the parser for syntax analysis

Other tasks of lexical analyzer

- One of the tasks of lexical analyzer is to strip out whitespace (blank, newline , tabs) and comments.
- Also, it can count the number of newline characters, so it give the line number corresponding to each error message.

Interaction with symbol table

- It is common for the lexical analyzer to interact with the symbol table as well.
- When the lexical analyzer discovers a lexeme constituting an identifier, it needs to enter that lexeme into the symbol table.

Interaction with symbol table and parser

- In some cases, information regarding the kind of identifier may be read from the symbol table by the lexical analyzer to assist it in determining the proper token it must pass to the parser

Lexical analyzer processes

- Sometimes, lexical analyzers are divided into a cascade of two processes:

A) Scanning

B) Lexical analysis

Lexical Analyzer

a) Scanning (with no tokenization) : deletion of comments and compaction of consecutive whitespace characters into one.

b) Lexical analysis (tokenization) : produces the sequence of tokens as output.

Tokens, Patterns, and Lexemes

- A token is a pair consisting of a token name and an optional attribute value.
- E

Example 3.2: The token names and associated attribute values for the Fortran statement

```
E = M * C ** 2
```

are written below as a sequence of pairs.

<**id**, pointer to symbol-table entry for E>
<**assign_op**>
<**id**, pointer to symbol-table entry for M>
<**mult_op**>
<**id**, pointer to symbol-table entry for C>
<**exp_op**>
<**number**, integer value 2>

Tokens, Patterns, and Lexemes

- A **pattern** is a description of the form that the lexemes of a token may take.
- In the case of a keyword as a token, the pattern is just the sequence of characters that form the keyword.
- For identifiers and some other tokens, the pattern is a more complex structure that is matched by many strings.
- A **lexeme** is a sequence of characters in the source program that matches the pattern for a token

Example
Tokens, Patterns, and Lexemes

TOKEN	INFORMAL DESCRIPTION	SAMPLE LEXEMES
if	characters `i`, `f`	`if`
else	characters `e`, `l`, `s`, `e`	`else`
comparison	`<` or `>` or `<=` or `>=` or `==` or `!=`	`<=`, `!=`
id	letter followed by letters and digits	`pi`, `score`, `D2`
number	any numeric constant	`3.14159`, `0`, `6.02e23`
literal	anything but `"`, surrounded by `"`'s	`"core dumped"`

Figure 3.2: Examples of tokens

Example

- Divide the following C++ program function:

```
float limitedSquare(float x)   {
    /* returns x-squared, but never more than 100 */
    return (x<=-10.0||x>=10.0)?100:x*x;
}
```

```
float limitedSquare(float x)   {
    /* returns x-squared, but never more than 100 */
    return (x<=-10.0||x>=10.0)?100:x*x;
}
```

<float> <id, limitedSquare> <(> <float> <id, x> <)> **<{>**

<return> <(> <id, x> <op,"<="> <num, -10.0> <op, "||"> <id, x> <op, ">="> <num, 10.0> <)> <op, "?"> <num, 100> <op, ":"> <id, x> <op, "*"> <id, x> **<}>**

Alphabet ,Strings and Languages

- An **alphabet** is any finite set of symbols. Typical examples of symbols are letters , digits, and punctuation.
- ASCII is an important example of an alphabet ; it is used in many software systems.
- A **string** over an alphabet is a finite sequence of symbols drawn from that alphabet.

Alphabet ,Strings and Languages

- In language theory, the terms "sentence" and "word" are often used as synonyms for "string." The length of a string s, usually written

  $|s|$, or Length(s) is the number of occurrences of symbols in s.

- For example, banana is a string of length six.
- The empty string, denoted ε or λ, is the string of length zero.
- A **language** is any countable set of strings over some fixed alphabet.
- Abstract languages like φ , the empty set, or {λ} , the set containing only the empty string, are languages under this definition.

Terms for Parts of Strings

- The following string-related terms are commonly used:
- 1 . A **prefix** of string s is any string obtained by removing zero or more symbols from the end of s. For example, ban, banana, and λ are prefixes of banana.
- 2. A **suffix** of string s is any string obtained by removing zero or more symbols from the beginning of s. For example, nana, banana, and λ are suffixes of banana.
- 3. A **substring** of s is obtained by deleting any prefix and any suffix from s. For instance, banana, nan, and λ are substrings of banana.

Terms for Parts of Strings

- 4. The proper prefixes, suffixes, and substrings of a string s are those, prefixes, suffixes, and substrings, respectively, of s that are not λ or not equal to s itself.
- 5. A **subsequence** of s is any string formed by deleting zero or more not necessarily consecutive positions of s. For example, baan is a subsequence of banana.

Concatenation of strings

- If x and y are strings, then the concatenation of x and y, denoted xy, is the string formed by appending y to x. For example, if **x = dog and y = house**, then **xy = doghouse**.
- The empty string is the identity under concatenation; that is, for any string s, λ s = s λ = s.
- If we think of concatenation as a product, we can define the "exponentiation“ of strings as follows. $s^0 = \lambda$, it follows that $s^1 = s$. Then $s^2 = ss$, $s^3 = sss$ and so on.

Operations on Languages

- In lexical analysis, the most important operations on languages are union, concatenation, and closure.
- Union is the familiar operation on sets.
- The concatenation of languages is all strings formed by taking a string from the first language and a string from the second language in all possible ways , and concatenating them.

Operations on Languages

- The (Kleene) closure of a language L, denoted **L^***, is the set of strings you get by **concatenating L (zero or more times).**
- Note that **L^0** , the "concatenation of L **zero times**," is defined to be {**λ**} ,
- the positive closure, denoted L^+ , is the same as the Kleene closure, but without the empty string ϵ.
- **L^+ denotes one or more repetitions**.

OPERATION	DEFINITION AND NOTATION
Union of L and M	$L \cup M = \{s \mid s \text{ is in } L \text{ or } s \text{ is in } M\}$
Concatenation of L and M	$LM = \{st \mid s \text{ is in } L \text{ and } t \text{ is in } M\}$
Kleene closure of L	$L^* = \cup_{i=0}^{\infty} L^i$
Positive closure of L	$L^+ = \cup_{i=1}^{\infty} L^i$

Figure 3.6: Definitions of operations on languages

Example

- Let L be the set of letters {A, B , . . . , Z , a, b, . . . , z} and let D be the set of digits {0, 1 , . . . 9}.
- We may think of L and D in two, essentially equivalent, ways. One way is that L and D are, respectively, the alphabets of uppercase and lowercase letters and of digits.
- The second way is that L and D are languages, all of whose strings happen to be of length one.

Example

- L^4 is the set of all 4-letter strings. Such as {abCD, aaaa, ….bxMN……. }
- L* is the set of all strings of letters, including ϵ, the empty string. {ϵ , *a, AB , aBC , mnO,…..,}*
- D^+ is the set of all strings of one or more digits. {0, 13, 346, 98765,…..}
- L(L U D)* is the set of all strings of letters and digits.

Regular Expressions

- A set is called regular if it can be generated from the elements of the alphabet using unio , concatenation and Kleene star operation.
- regular expressions has come into common use for describing all the languages that can be built from regular sets.

Example 3.4: Let $\Sigma = \{a, b\}$.

1. The regular expression **a|b** denotes the language $\{a, b\}$.

2. **(a|b)(a|b)** denotes $\{aa, ab, ba, bb\}$, the language of all strings of length two over the alphabet Σ. Another regular expression for the same language is **aa|ab|ba|bb**.

3. $\mathbf{a}^*$ denotes the language consisting of all strings of zero or more a's, that is, $\{\epsilon, a, aa, aaa, \ldots\}$.

4. $(\mathbf{a}|\mathbf{b})^*$ denotes the set of all strings consisting of zero or more instances of a or b, that is, all strings of a's and b's: $\{\epsilon, a, b, aa, ab, ba, bb, aaa, \ldots\}$. Another regular expression for the same language is $(\mathbf{a}^*\mathbf{b}^*)^*$.

5. $\mathbf{a}|\mathbf{a}^*\mathbf{b}$ denotes the language $\{a, b, ab, aab, aaab, \ldots\}$, that is, the string a and all strings consisting of zero or more a's and ending in b.

Thank you

